অধ্যায় ১৮

১৮

Functional Programming, Immutability এবং Advanced Data Transformation

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Referential Transparency ব্যাখ্যা করতে, এবং কেন তা Code-কে যুক্তি দিয়ে বোঝা সহজ করে তা বলতে।
- Nested তথ্য Immutable উপায়ে হালনাগাদ করতে, এবং Structural Sharing-এর সুবিধা কাজে লাগাতে।
- Path ধরে হালনাগাদের একটি সাধারণ সাহায্যকারী (`setIn` , `updateIn`) লিখতে।
- `flatMap` দিয়ে Nested তথ্য সমতল করতে, এবং `reduce` দিয়ে `Map` -এ জমাতে।
- `Map.groupBy` ও বহুস্তর Grouping দিয়ে Aggregation ও Pivot তৈরি করতে।
- একাধিক তালিকা ID ধরে জোড়া দিতে (Join), দক্ষ উপায়ে।
- Stable, বহু-Key তুলনা এবং বাংলা নামের জন্য `Intl.Collator` দিয়ে সাজাতে।
- Result Type দিয়ে ভুল সারিসহ তথ্যের প্রবাহ সামলাতে।
- Asynchronous তথ্য-প্রবাহ ক্রমে, একসঙ্গে, বা সীমিত দলে চালাতে।
- একটি সম্পূর্ণ **LedgerFlow** Order Ledger Analytics তৈরি ও পরীক্ষা করতে।

পূর্বশর্ত

- অধ্যায় ৬–৭: Array Method, Object, Spread, Shallow বনাম Deep Copy।
- অধ্যায় ১০: Promise, `Promise.all` , সীমিত Concurrency (অনুশীলনী ১১)।
- অধ্যায় ১২: `toSorted` , `structuredClone` , `Object.groupBy` ।
- অধ্যায় ১৪: Pure Function, Side Effect, `pipe` , `compose` , Currying, Memoization।

এই অধ্যায় কেন গুরুত্বপূর্ণ

অধ্যায় ১৪-এ Functional Programming-এর মূল হাতিয়ারগুলো শিখেছেন: Pure Function, Composition, Currying। এই অধ্যায় সেগুলোকে বাস্তব তথ্যের মুখোমুখি করে। বাস্তব তথ্য সমতল তালিকা নয়: একটি Order-এর ভেতরে একাধিক পণ্য, প্রতিটি পণ্যের তথ্য আরেকটি তালিকায়, গ্রাহকের তথ্য তৃতীয়টিতে; কিছু সারি ভাঙা; কিছু তথ্য Network থেকে আসে, সময় নিয়ে। আর এই তথ্য বদলাতেও হয়: একটি Order ফেরত এলে তার একটি পণ্যের অবস্থা বদলাতে হবে, পুরো ব্যবস্থাকে অক্ষত রেখে।

এগুলো সামলানোর Functional উপায়গুলো আধুনিক Frontend (React-এর State, Redux) ও Data Processing-এর মূল ভিত্তি। এই অধ্যায় শেষে আপনি জটিল, Nested, অসম্পূর্ণ তথ্যকে নির্ভরযোগ্য, পরীক্ষাযোগ্য Pipeline-এ রূপান্তর করতে পারবেন।

১৮.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা	এই অধ্যায়ে নতুন
Pure Function, Side Effect (অধ্যায় ১৪)	Referential Transparency
Spread দিয়ে এক স্তরের Copy (অধ্যায় ৭)	Nested হালনাগাদ, Structural Sharing, `setIn` / `updateIn`
`map`, `filter`, `reduce` (অধ্যায় ৬)	`flatMap`, `reduce`-এ `Map`
`Object.groupBy` (অধ্যায় ১২)	`Map.groupBy`, বহুস্তর Grouping, Pivot
`localeCompare` (অধ্যায় ৮)	Stable Sort, বহু-Key Comparator, `Intl.Collator`
`partitionValid` (অধ্যায় ১৪)	Result Type
`Promise.all` (অধ্যায় ১০)	Asynchronous Pipeline, Batch প্রক্রিয়া

১৮.২ Referential Transparency

একটি Expression **Referentially Transparent** যদি তাকে তার ফল দিয়ে প্রতিস্থাপন করলে Program-এর আচরণ বদলায় না। Pure Function-এর ডাক এমনই: `add(2, 3)` সর্বত্র `5` দিয়ে বদলানো যায়।

transparency.js · JavaScript — চলবে: Browser Console ও Node.js

```
const vat = (price) => Math.round(price * 1.15);

const total1 = vat(1000) + vat(1000);
const once = vat(1000);
const total2 = once + once;
console.log(total1 === total2);

let counter = 0;
```

```
const nextId = () => {
  counter += 1;
  return counter;
};

const ids1 = [nextId(), nextId()];
const single = nextId();
const ids2 = [single, single];
console.log(ids1.join(','), ids2.join(','));
```

Output

```
true
1,2 3,3
```

`vat(1000)` দুবার ডাকা বা একবার ডেকে দুবার ব্যবহার—একই ফল। `nextId()` তেমন নয়: একবার ডেকে মান পুনর্ব্যবহার করলে আচরণ বদলে গেছে। এর ব্যবহারিক মূল্য: Referentially Transparent Code-এ আপনি নিশ্চিন্তে Refactor করতে পারেন (একটি হিসাব একটি Variable-এ তুলে নেওয়া, Memoize করা, ক্রম বদলানো), কারণ ফল বদলাবে না। Impure Code-এ এমন প্রতিটি পরিবর্তন একটি সম্ভাব্য Bug।

১৮.৩ Nested তথ্যের Immutable হালনাগাদ

অধ্যায় ৭-এ এক স্তরের Spread দেখেছেন। Nested তথ্যে প্রতিটি স্তরে নতুন Object তৈরি করতে হয়, **শুধু বদলের পথ ধরে**।

nested-update.js · JavaScript চলবে: Browser Console ও Node.js

```
const store = {
  name: 'Boi Ghar',
  address: { city: 'Bogura', zip: '5800' },
  orders: [
    { id: 'O1', items: [{ sku: 'B1', qty: 2 }, { sku: 'B2', qty: 1 }] },
    { id: 'O2', items: [{ sku: 'B3', qty: 5 }] },
  ],
};

const updated = {
  ...store,
  orders: store.orders.map((order) =>
    (order.id !== 'O1' ? order : {
      ...order,
      items: order.items.map((item) =>
        (item.sku === 'B2' ? { ...item, qty: 3 } : item)),
    })),
};

console.log(store.orders[0].items[1].qty, updated.orders[0].items[1].qty);
```

```
console.log(updated.address === store.address);
console.log(updated.orders[1] === store.orders[1]);
console.log(updated.orders[0].items[0] === store.orders[0].items[0]);
console.log(updated.orders[0] === store.orders[0]);
```

Output

```
1 3
true
true
true
false
```

শুধু বদলের পথের Object নতুন: মূল Store, `orders` Array, `O1` Order, তার `items` Array এবং `B2` পণ্য। বাকি সব— `address`, `O2`, `B1` —**একই** Object, নতুন ও পুরোনো Store দুজনেই ভাগ করে নেয়। একে বলে **Structural Sharing**।

এর দুটি বড় সুবিধা:

- **সস্তা:** ১০,০০০ Order-এর Store-এ একটি পণ্য বদলাতে পাঁচটি Object তৈরি হয়, ১০,০০০ নয়। `structuredClone` দিয়ে পুরোটা Copy করলে সব Copy হতো।
- **দ্রুত পরিবর্তন-শনাক্তকরণ:** কোন অংশ বদলেছে জানতে গভীর তুলনা লাগে না; `===` যথেষ্ট। `updated.address === store.address` সত্য মানে ঠিকানা বদলায়নি। React এই নীতিতেই কোন অংশ নতুন করে আঁকতে হবে তা ঠিক করে।

তবে লক্ষ করুন, হাতে লেখা Nested Spread পড়া কষ্টকর এবং ভুলপ্রবণ।

Path ধরে হালনাগাদ: setIn এবং updateIn

update-in.js · JavaScript চলবে: Browser Console ও Node.js

```
function updateIn(target, [key, ...rest], change) {
  const current = target?.[key];
  const next = rest.length === 0
    ? change(current)
    : updateIn(current, rest, change);
  if (Array.isArray(target)) {
    return target.with(key, next);
  }
  return { ...target, [key]: next };
}

const setIn = (target, path, value) => updateIn(target, path, () => value);

const store = { address: { city: 'Bogura' }, orders: [{ items: [{ qty: 2 }] }] };
const a = setIn(store, ['address', 'city'], 'Dhaka');
const b = updateIn(store, ['orders', 0, 'items', 0, 'qty'], (qty) => qty + 1);
```

```
console.log(a.address.city, store.address.city, a.orders === store.orders);
console.log(b.orders[0].items[0].qty, b.address === store.address);
```

Output

```
Dhaka Bogura true
3 true
```

`updateIn` Recursive: Path-এর প্রথম Key-র মানকে বাকি Path দিয়ে হালনাগাদ করে, তারপর সেই স্তরের একটি নতুন Object (বা Array-র জন্য `with`, অধ্যায় ১২) ফেরত দেয়। Structural Sharing নিজে থেকেই রক্ষিত। Immer-এর মতো Library একই কাজ আরও সুবিধাজনক Syntax-এ করে; ভেতরের নীতিটি এটিই।

Deep Freeze: ভুল করে বদলানো ধরা

`Object.freeze` শুধু এক স্তরের (অধ্যায় ৭)। Development-এ পুরো গাছ Freeze করলে ভুল করে Mutation সঙ্গে সঙ্গে ধরা পড়ে:

deep-freeze.js · JavaScript — চলবে: Browser Console ও Node.js

```
'use strict';

function deepFreeze(value) {
  if (value !== null && typeof value === 'object' && !Object.isFrozen(value)) {
    Object.freeze(value);
    Object.values(value).forEach(deepFreeze);
  }
  return value;
}

const config = deepFreeze({ tax: { rate: 15 }, regions: ['Dhaka', 'Sylhet'] });

try {
  config.tax.rate = 0;
} catch (error) {
  console.log(error.name);
}
try {
  config.regions.push('Khulna');
} catch (error) {
  console.log(error.name);
}
console.log(config.tax.rate, config.regions.length);
```

Output

```
TypeError
TypeError
15 2
```

Strict Mode-এ Frozen Object বদলানোর চেষ্টা `TypeError` দেয় (Strict Mode ছাড়া নীরবে উপেক্ষিত হয়)। Freeze Production-এ সাধারণত বাদ রাখা হয়, কারণ গভীর গাছ Freeze করতে সময় লাগে; Test ও Development-এ এটি একটি সস্তা সুরক্ষা। Project-এর Test সব ইনপুট Deep Freeze করে চালানো হয়।

কখন Mutation গ্রহণযোগ্য

Immutability একটি নীতি, ধর্ম নয়। মূল প্রশ্ন হলো **মালিকানা**: যে Object শুধু একটি Function-এর ভেতরে তৈরি হয়েছে এবং এখনো বাইরে যায়নি, সেটিকে বদলানো নিরাপদ, কারণ আর কেউ তাকে দেখছে না। অধ্যায় ১৪-এর `partitionValid` এবং এই অধ্যায়ের `reduce`-এর `Map` এর উদাহরণ। কিন্তু যে Object Argument হিসেবে এসেছে, অথবা ফেরত দেওয়ার পর অন্য কেউ রেখে দিয়েছে, তা কখনো বদলাবেন না।

একটি দ্বিতীয় ব্যতিক্রম পারফরম্যান্স: লক্ষাধিক সারির একটি Loop-এ প্রতিটি ধাপে নতুন Object তৈরি Garbage Collector-এর ওপর চাপ ফেলে। তখন একটি স্থানীয় Array-তে `push` করে শেষে একবার ফেরত দেওয়াই যুক্তিসঙ্গত। কিন্তু এই সিদ্ধান্ত নেওয়ার আগে মাপুন (অধ্যায় ১৯); সাধারণ আকারের তথ্যে পার্থক্য চোখে পড়ে না, অথচ Mutation-এর Bug খুঁজতে দিন লেগে যায়।

তথ্যের আকার: Nested বনাম Normalized

একই তথ্য দুইভাবে রাখা যায়। **Nested** আকারে প্রতিটি Order-এর ভেতরে গ্রাহকের পুরো তথ্য; **Normalized** আকারে প্রতিটি ধরনের সত্তা (Entity) আলাদা, ID ধরে, এবং সম্পর্ক শুধু ID দিয়ে—Database-এর সারণির মতো।

normalized.js · JavaScript চলবে: Node.js

```
const nested = [
  { id: 'O1', customer: { id: 'C1', name: 'Sara', city: 'Bogura' }, total: 500 },
  { id: 'O2', customer: { id: 'C1', name: 'Sara', city: 'Bogura' }, total: 300 },
];

function normalize(orders) {
  const customers = {};
  const byId = {};
  for (const { customer, ...order } of orders) {
    customers[customer.id] = customer;
    byId[order.id] = { ...order, customerId: customer.id };
  }
  return { orders: byId, customers };
}

const state = normalize(nested);
```

```
const moved = {
  ...state,
  customers: { ...state.customers, C1: { ...state.customers.C1, city: 'Dhaka' } },
};

const cityOf = (s, orderId) => s.customers[s.orders[orderId].customerId].city;
const count = Object.keys(state.customers).length;
console.log(count, cityOf(moved, 'O1'), cityOf(moved, 'O2'));
console.log(moved.orders === state.orders);
```

Output

```
1 Dhaka Dhaka
true
```

Nested আকারে Sara-র শহর দুটি জায়গায়; ঠিকানা বদলালে দুটিই বদলাতে হতো, একটি ভুলে গেলে তথ্য পরস্পরবিরোধী। Normalized আকারে তথ্য একটি জায়গায়: একটি হালনাগাদে দুটি Order-ই নতুন শহর দেখায়, এবং Order-এর তালিকা অক্ষত (`===`)। অনেক সত্তা ও পারস্পরিক সম্পর্কের Application State-এ (Redux-এর নির্দেশিকাও তাই বলে) Normalized আকার প্রায় সব সময় ভালো; দেখানোর সময় প্রয়োজনমতো জোড়া দেওয়া হয় (অনুচ্ছেদ ১৮.৬-এর Join)।

১৮.৪ flatMap এবং reduce-এর গভীরে

একটি Order-এর ভেতরে অনেক পণ্য; বিক্রির হিসাবের জন্য দরকার প্রতিটি পণ্যের একটি সমতল তালিকা।

flat-map.js · JavaScript চলবে: Node.js

```
const orders = [
  { id: 'O1', customer: 'C1', items: [
    { sku: 'B1', qty: 2 }, { sku: 'B2', qty: 1 },
  ] },
  { id: 'O2', customer: 'C2', items: [] },
  { id: 'O3', customer: 'C1', items: [{ sku: 'B1', qty: 1 }] },
];

const lines = orders.flatMap((order) =>
  order.items.map((item) =>
    ({ order: order.id, customer: order.customer, ...item })));

console.log(lines.length);
const labels = lines.map((line) => `${line.order}:${line.sku}x${line.qty}`);
console.log(labels.join(' '));

const words = ['আম কলা', 'লিচু'].flatMap((text) => text.split(' '));
console.log(words);
```

Output

```
3
O1:B1x2 O1:B2x1 O3:B1x1
[ 'আম', 'কলা', 'লিচু' ]
```

`flatMap(fn)` মানে `map(fn).flat()`: প্রতিটি উপাদান থেকে একটি Array, তারপর সব জোড়া লাগিয়ে একটি সমতল তালিকা। পণ্যহীন `O2` থেকে ফাঁকা Array, তাই সে নিঃশব্দে বাদ। একটি উপাদান থেকে শূন্য, এক বা একাধিক ফল—এটিই `flatMap`-এর শক্তি।

`reduce` দিয়ে একটি `Map`-এ জমানো, Key যেকোনো ধরনের:

reduce-map.js · JavaScript চলবে: Node.js

```
const lines = [
  { sku: 'B1', qty: 2 }, { sku: 'B2', qty: 1 }, { sku: 'B1', qty: 1 },
];

const qtyBySku = lines.reduce(
  (totals, line) => totals.set(line.sku, (totals.get(line.sku) ?? 0) + line.qty),
  new Map(),
);

console.log(qtyBySku);
console.log([...qtyBySku].toSorted((a, b) => b[1] - a[1])[0]);
```

Output

```
Map(2) { 'B1' => 3, 'B2' => 1 }
[ 'B1', 3 ]
```

`Map.set` নিজেই Map ফেরত দেয়, তাই Reducer এক লাইনে। এখানে Reducer-এর ভেতরে একটি নতুন Map বদলানো হচ্ছে, যা `reduce`-এর বাইরে কেউ দেখে না; বাইরে থেকে পুরো হিসাব Pure (অধ্যায় ১৪-এর পারফরম্যান্স-ফাঁদের সমাধানও এটিই)। `[...map]` একটি Map থেকে `[key, value]` জোড়ার Array দেয়।

১৮.৫ Grouping, Aggregation এবং Pivot

`Object.groupBy` (অধ্যায় ১২) Key-কে লেখায় রূপান্তর করে। **`Map.groupBy`** Key-কে যেমন আছে রাখে—সংখ্যা, Object, এমনকি তারিখও।

group-by.js · JavaScript চলবে: Node.js

```
const sales = [
  { region: 'Dhaka', month: '2026-01', amount: 1200 },
  { region: 'Sylhet', month: '2026-01', amount: 800 },
```

```
  { region: 'Dhaka', month: '2026-02', amount: 500 },
  { region: 'Dhaka', month: '2026-01', amount: 300 },
];

const band = (sale) => (sale.amount >= 1000 ? 1000 : 0);
const byAmountBand = Map.groupBy(sales, band);
console.log([...byAmountBand.keys()], typeof [...byAmountBand.keys()][0]);

function aggregate(rows) {
  return rows.reduce(
    (stats, row) => ({
      count: stats.count + 1,
      total: stats.total + row.amount,
      max: Math.max(stats.max, row.amount),
    }),
    { count: 0, total: 0, max: -Infinity },
  );
}

const pivot = {};
const byRegion = Object.groupBy(sales, (s) => s.region);
for (const [region, rows] of Object.entries(byRegion)) {
  pivot[region] = Object.fromEntries(
    Object.entries(Object.groupBy(rows, (s) => s.month))
      .map(([month, monthRows]) => [month, aggregate(monthRows).total]),
  );
}
console.log(pivot);
console.log(aggregate(sales));
```

Output

```
[ 1000, 0 ] number
{
  Dhaka: { '2026-01': 1500, '2026-02': 500 },
  Sylhet: { '2026-01': 800 }
}
{ count: 4, total: 2800, max: 1200 }
```

- `Map.groupBy`-এর Key `1000` একটি সংখ্যা রয়ে গেছে; `Object.groupBy` হলে তা `'1000'` লেখা হতো।
- **একবারে একাধিক মান (`aggregate`):** একটি `reduce` দিয়ে সংখ্যা, মোট ও সর্বোচ্চ। Accumulator একটি ছোট নতুন Object; ছোট বলে প্রতিবার নতুন তৈরি করা সস্তা।
- **Pivot:** বহুস্তর Grouping—প্রথমে অঞ্চল, তারপর প্রতিটি অঞ্চলের ভেতরে মাস। এটিই Spreadsheet-এর Pivot Table: সারি অঞ্চল, কলাম মাস। `Object.fromEntries` জোড়ার তালিকা থেকে Object তৈরি করে, `Object.entries`-এর উল্টো।

১৮.৬ Join: তালিকা জোড়া দেওয়া

বিক্রির সারিতে শুধু পণ্যের `sku`; দাম ও নাম আরেকটি তালিকায়। প্রতিটি সারির জন্য `products.find(...)` লিখলে প্রতিটি সারিতে পুরো পণ্য-তালিকা খোঁজা হয়: ১০,০০০ সারি ও ১,০০০ পণ্যে প্রায় এক কোটি তুলনা। আগে একটি **Index** (`Map`) তৈরি করলে প্রতিটি খোঁজা প্রায় তাৎক্ষণিক (অধ্যায় ১৯-এ এর বিশ্লেষণ)।

join.js · JavaScript — চলবে: Node.js

```
const products = [
  { sku: 'B1', name: 'Notebook', price: 120 },
  { sku: 'B2', name: 'Pen Set', price: 250 },
];
const lines = [
  { order: 'O1', sku: 'B1', qty: 2 },
  { order: 'O1', sku: 'B9', qty: 1 },
  { order: 'O2', sku: 'B2', qty: 1 },
];

const indexBy = (rows, key) => new Map(rows.map((row) => [row[key], row]));
const productIndex = indexBy(products, 'sku');

const joined = lines.flatMap((line) => {
  const product = productIndex.get(line.sku);
  if (!product) {
    return [];
  }
  return [{ ...line, name: product.name, total: product.price * line.qty }];
});
const missing = lines
  .filter((line) => !productIndex.has(line.sku))
  .map((line) => line.sku);

console.log(joined.map((row) => `${row.order} ${row.name} ${row.total}`));
console.log(missing);
```

Output

```
[ 'O1 Notebook 240', 'O2 Pen Set 250' ]
[ 'B9' ]
```

এটি Database-এর **Inner Join**: দুই পাশে মেলে এমন সারিগুলোই ফলে। `flatMap`-এ মেলেনি এমন সারির জন্য ফাঁকা Array বাদ দেওয়ার কৌশল লক্ষ করুন। অচেনা `B9` নিঃশব্দে হারিয়ে যায়নি, আলাদা তালিকায় জানানো হয়েছে—তথ্যের মান যাচাইয়ে এটি অপরিহার্য।

১৮.৭ Sorting: Stability, বহু Key এবং বাংলা

JavaScript-এর `sort` ও `toSorted` **Stable** (ES2019 থেকে নিশ্চিত): দুটি উপাদান সমান হলে তাদের মূল ক্রম অক্ষত থাকে। তাই একাধিক Key-তে সাজাতে Comparator জোড়া দেওয়া যায়:

sorting.js · JavaScript — চলবে: Node.js

```
const by = (key, direction = 1) => (a, b) =>
  (a[key] < b[key] ? -direction : a[key] > b[key] ? direction : 0);
const thenBy = (...comparators) => (a, b) => {
  for (const compare of comparators) {
    const result = compare(a, b);
    if (result !== 0) {
      return result;
    }
  }
  return 0;
};

const students = [
  { name: 'Sara', class: 9, marks: 88 },
  { name: 'Rafi', class: 10, marks: 88 },
  { name: 'Mina', class: 9, marks: 92 },
  { name: 'Nila', class: 9, marks: 88 },
];

const order = thenBy(by('class'), by('marks', -1), by('name'));
const ranked = students.toSorted(order);
console.log(ranked.map((s) => `${s.class}-${s.name}`).join(' '));

const collator = new Intl.Collator('bn');
console.log(['সুমন', 'আরিফ', 'ঋতু', 'কবির'].toSorted(collator.compare));
const natural = new Intl.Collator('en', { numeric: true });
console.log(['item10', 'item2', 'item1'].toSorted(natural.compare));
```

Output

```
9-Mina 9-Nila 9-Sara 10-Rafi
[ 'আরিফ', 'ঋতু', 'কবির', 'সুমন' ]
[ 'item1', 'item2', 'item10' ]
```

- **`thenBy`** Comparator-এর একটি Composition (অধ্যায় ১৪): প্রথমটি সমান বললে পরেরটি। শ্রেণি অনুযায়ী, তারপর নম্বর (বেশি আগে), তারপর নাম।
- **`Intl.Collator`** ভাষার নিয়মে তুলনা করে, এবং বারবার তুলনায় `localeCompare`-এর চেয়ে দ্রুত, কারণ নিয়ম একবারই তৈরি হয়। বাংলা স্বরবর্ণ ও ব্যঞ্জনবর্ণ অভিধানের ক্রমে এসেছে।
- **`numeric: true`**: লেখার ভেতরের সংখ্যাকে সংখ্যা হিসেবে তুলনা— `item2` আগে `item10`-এর।

১৮.৮ Result Type: ভুলসহ তথ্যের প্রবাহ

অধ্যায় ১৪-এর `partitionValid` বৈধ ও অবৈধ সারি আলাদা করেছিল। একটি সাধারণ রূপ হলো প্রতিটি ধাপ একটি **Result** ফেরত দেবে— `{ ok: true, value }` বা `{ ok: false, error }` —এবং Pipeline-এর ধাপগুলো Result-এর ওপর কাজ করবে।

result.js · JavaScript — চলবে: Node.js

```
const ok = (value) => ({ ok: true, value });
const fail = (error) => ({ ok: false, error });
const andThen = (fn) => (result) => (result.ok ? fn(result.value) : result);

const parseQty = (text) =>
  (/^\d+$/.test(text) ? ok(Number(text)) : fail(`পরিমাণ সংখ্যা নয়: "${text}"`));
const positive = (n) => (n > 0 ? ok(n) : fail('পরিমাণ শূন্য'));
const withinStock = (n) => (n <= 50 ? ok(n) : fail(`স্টকে ${n}টি নেই`));

const checkQty = (text) => [andThen(positive), andThen(withinStock)]
  .reduce((result, step) => step(result), parseQty(text));

for (const input of ['12', 'abc', '0', '80']) {
  const result = checkQty(input);
  console.log(input, result.ok ? result.value : result.error);
}
```

Output

```
12 12
abc পরিমাণ সংখ্যা নয়: "abc"
0 পরিমাণ শূন্য
80 স্টকে ৪০টি নেই
```

`andThen` একটি সফল Result-এর মান পরের ধাপে পাঠায়, আর ব্যর্থ Result-কে অপরিবর্তিত পেরিয়ে যেতে দেয়। ফলে প্রথম ব্যর্থতার পরের ধাপগুলো নিজে থেকেই বাদ পড়ে, `try` / `catch` বা `if` ছাড়া। Exception-এর সঙ্গে পার্থক্য (অধ্যায় ১১-এর সমাধান ১২): ব্যর্থতা এখানে একটি সাধারণ মান, তাই হাজারো সারির তালিকায় প্রতিটির ফল জমিয়ে রাখা, গোনা বা Report করা সহজ।

১৮.৯ Asynchronous Pipeline

তথ্য যখন File, Database বা Network থেকে আসে, Pipeline-এর ধাপ Asynchronous। তিনটি ধরন:

async-pipeline.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));
let active = 0;
```

```
let peak = 0;

async function loadFile(name) {
  active += 1;
  peak = Math.max(peak, active);
  await wait(30);
  active -= 1;
  return `${name}: ১০০ সারি`;
}

const files = ['jan', 'feb', 'mar', 'apr', 'may'];

const sequential = [];
for (const file of files) {
  sequential.push(await loadFile(file));
}
console.log(`ক্রমে: সর্বোচ্চ একসঙ্গে ${peak}`);

peak = 0;
const all = await Promise.all(files.map(loadFile));
console.log(`একসঙ্গে: সর্বোচ্চ একসঙ্গে ${peak}, ফল ${all.length}`);

peak = 0;
const batched = [];
for (let i = 0; i < files.length; i += 2) {
  const batch = files.slice(i, i + 2);
  batched.push(...(await Promise.all(batch.map(loadFile))));
}
console.log(`দলে দলে: সর্বোচ্চ একসঙ্গে ${peak}, ফল ${batched.length}`);
console.log(batched[4]);
```

Output

```
ক্রমে: সর্বোচ্চ একসঙ্গে 1
একসঙ্গে: সর্বোচ্চ একসঙ্গে 5, ফল 5
দলে দলে: সর্বোচ্চ একসঙ্গে 2, ফল 5
may: ১০০ সারি
```

ধরন	সুবিধা	ঝুঁকি
ক্রমে (`for...of` + `await`)	সরল; প্রতিটি ধাপ আগেরটির ফল ব্যবহার করতে পারে	ধীর
একসঙ্গে (`Promise.all`)	দ্রুততম	অনেক কাজে Server বা Memory-র ওপর চাপ
দলে দলে (Batch)	গতি ও চাপের ভারসাম্য	একটি ধীর কাজ পুরো দলকে আটকায়

অধ্যায় ১০-এর `runLimited` (Worker Pool) দলে দলের চেয়েও দক্ষ: একটি কাজ শেষ হলেই পরেরটি শুরু হয়। Asynchronous Pipeline-এ একটি সীমাবদ্ধতা মনে রাখুন: `map`, `filter`, `reduce` Promise-এর জন্য অপেক্ষা করে না (অধ্যায় ১০-এর `forEach` ফাঁদ)। `filter(async ...)` সব উপাদান রেখে দেয়, কারণ প্রতিটি Promise Truthy।

১৮.১০ সাধারণ ভুল

ভুল	কী দেখা যায়	সমাধান
এক স্তরের Spread দিয়ে Nested হালনাগাদ	মূল তথ্যের ভেতরের Object বদলে যায়	পথের প্রতিটি স্তরে নতুন Object, বা `updateIn`
সব কিছু `structuredClone` করে হালনাগাদ	ধীর; `===` দিয়ে পরিবর্তন বোঝা যায় না	Structural Sharing
`map` দিয়ে Nested Array, তারপর ভুলে যাওয়া `flat`	Array-র ভেতরে Array	`flatMap`
প্রতিটি সারিতে `find` দিয়ে Join	বড় তথ্যে অত্যন্ত ধীর	আগে `Map` Index
Join-এ না মেলা সারি নিঃশব্দে বাদ	তথ্য হারায়	আলাদা তালিকায় জানান
`Object.groupBy`-এর Key সংখ্যা ভাবা	Key লেখা হয়ে যায়	`Map.groupBy`
বাংলা নাম `<` দিয়ে সাজানো	অভিধানের ক্রম নয়	`Intl.Collator('bn')`
Comparator সমান ক্ষেত্রে `0` দেয় না	অনিশ্চিত ক্রম	তিন ক্ষেত্রই সামলান
`filter(async ...)`	কিছুই বাদ পড়ে না	আগে `Promise.all`, তারপর `filter`
সীমাহীন `Promise.all`	Server-এ অনুরোধের বন্যা	Batch বা Worker Pool

১৮.১১ Debugging নির্দেশিকা

- **অপ্রত্যাশিত Mutation ধরতে** Test-এ ইনপুট Deep Freeze করুন; বদলানোর চেষ্টার লাইনেই `TypeError`।
- **কোন অংশ বদলেছে?** পুরোনো ও নতুন অবস্থার প্রতিটি শাখা `===` দিয়ে তুলনা করুন; অপ্রত্যাশিত `false` মানে অকারণে নতুন Object।
- **Pipeline-এর মাঝে দেখুন** অধ্যায় ১৪-এর `tap` দিয়ে, বিশেষত Join ও Grouping-এর পর সারির সংখ্যা।
- **সংখ্যা মেলে না?** মোট হিসাব দুইভাবে করুন (সব সারির যোগফল এবং দলগুলোর যোগফলের যোগফল); দুটি না মিললে কোনো সারি হারিয়েছে বা দুবার গোনা হয়েছে।
- **`console.table(rows)`** সারির তালিকা সারণি আকারে দেখায়।

- **Asynchronous ক্রম অপ্রত্যাশিত?** প্রতিটি ধাপের শুরু ও শেষ Log করুন, একসঙ্গে কতগুলো চলছে গুনুন (উদাহরণের `peak`-এর মতো)।

১৮.১২ পেশাদার অভ্যাস

- **তথ্য কখনো জায়গায় বদলাবেন না;** নতুন অবস্থা, Structural Sharing-সহ।
- **Test-এ ইনপুট Deep Freeze।**
- **Normalize আগে, তারপর Join ও Aggregation,** যাতে প্রতিটি ধাপ পরিষ্কার তথ্য পায়।
- **Join-এর আগে Index;** না মেলা সারি জানান।
- **প্রতিটি ধাপ ছোট, নামযুক্ত, আলাদাভাবে পরীক্ষাযোগ্য।**
- **মোট হিসাবের Invariant যাচাই** (অধ্যায় ১৬): দলের যোগফল = সব সারির যোগফল।
- **সাজানোয় সব সময় শেষ Key হিসেবে একটি অনন্য ID,** যাতে ক্রম নিশ্চিত।
- **Asynchronous কাজে সীমিত Concurrency।**

১৮.১৩ Project: LedgerFlow

একটি অনলাইন বইয়ের দোকানের Order Ledger থেকে Report: কাঁচা Order (প্রতিটিতে একাধিক পণ্য), পণ্যের তালিকা এবং গ্রাহকের তালিকা যাচাই ও জোড়া দিয়ে অঞ্চল ও মাসভিত্তিক Pivot, শীর্ষ গ্রাহক, এবং ফেরত (Refund) সংশোধন—সবই Immutable উপায়ে। তথ্য একাধিক মাসিক "File" থেকে Asynchronous ভাবে আসে, সীমিত দলে।

অধ্যায় ১৪-এর Sales Function Lab একটি সমতল তালিকার হিসাব ছিল। LedgerFlow-এর তথ্য Nested ও একাধিক উৎসের, এবং তাতে সংশোধন হয়।

Project-এর চাহিদা

- কাঁচা Order যাচাই (Result Type): ID, গ্রাহক, তারিখ (`YYYY-MM-DD`), কমপক্ষে একটি পণ্য, প্রতিটি পণ্যের ধনাত্মক পূর্ণসংখ্যা পরিমাণ।
- Order-কে পণ্য-সারিতে সমতল করা (`flatMap`); পণ্য ও গ্রাহকের সঙ্গে Index-ভিত্তিক Join; অচেনা পণ্য বা গ্রাহক জানানো।
- অঞ্চল × মাস Pivot; গ্রাহকভিত্তিক মোট; শীর্ষ গ্রাহক (সমান হলে নাম, বাংলা নিয়মে)।
- Refund: একটি Order-এর একটি পণ্যের ফেরত পরিমাণ চিহ্নিত করা, Immutable, Structural Sharing-সহ।
- মাসিক File Asynchronous Loader থেকে, সর্বোচ্চ ২টি একসঙ্গে; একটি File ব্যর্থ হলে বাকিগুলোর ফলসহ ব্যর্থতার তালিকা।
- Invariant: Pivot-এর সব ঘরের যোগফল = সব সারির মোট।

File-এর কাঠামো

Folder-এর কাঠামো

```
ledger-flow/
├── src/
│   ├── immutable.mjs     updateIn, setIn, deepFreeze
│   ├── validate.mjs      Result Type এবং Order যাচাই
│   ├── transform.mjs     flatten, Index, Join, Pivot, শীর্ষ গ্রাহক, Refund
│   └── load.mjs          Asynchronous Batch Loader
├── data/
│   └── sample.mjs        নমুনা তথ্য ও কৃত্রিম Loader
├── report.mjs
├── README.md
└── tests/
    ├── immutable.test.mjs
    ├── transform.test.mjs
    └── load.test.mjs
```

src/immutable.mjs ও src/validate.mjs

immutable.mjs · JavaScript চলবে: Browser Console ও Node.js

```javascript
export function updateIn(target, [key, ...rest], change) {
  const current = target?.[key];
  const next = rest.length === 0
    ? change(current)
    : updateIn(current, rest, change);
  if (Object.is(next, current)) {
    return target;
  }
  if (Array.isArray(target)) {
    return target.with(key, next);
  }
  return { ...target, [key]: next };
}

export const setIn = (target, path, value) => updateIn(target, path, () => value);

export function deepFreeze(value) {
  if (value !== null && typeof value === 'object' && !Object.isFrozen(value)) {
    Object.freeze(value);
    Object.values(value).forEach(deepFreeze);
  }
  return value;
}
```

অধ্যায়ের সংস্করণের সঙ্গে একটি পার্থক্য: নতুন মান পুরোনোটির সমান হলে (`Object.is`) কোনো নতুন Object তৈরি হয় না, পুরোনোটিই ফেরত। ফলে "কিছু না বদলানো" হালনাগাদ `===` দিয়ে শনাক্ত করা যায়।

validate.mjs · JavaScript — চলবে: Browser Console ও Node.js

```
export const ok = (value) => ({ ok: true, value });
export const fail = (error) => ({ ok: false, error });

const DATE = /^\d{4}-(0[1-9]|1[0-2])-(0[1-9]|[12]\d|3[01])$/;
const isText = (value) => typeof value === 'string' && value.trim() !== '';

export function validateOrder(raw) {
  if (!isText(raw?.id)) {
    return fail('Order ID নেই');
  }
  if (!isText(raw.customer)) {
    return fail(`${raw.id}: গ্রাহক নেই`);
  }
  if (!DATE.test(raw.date ?? '')) {
    return fail(`${raw.id}: তারিখ YYYY-MM-DD নয়`);
  }
  if (!Array.isArray(raw.items) || raw.items.length === 0) {
    return fail(`${raw.id}: কোনো পণ্য নেই`);
  }
  const badItem = raw.items.find((item) =>
    !isText(item?.sku) || !Number.isInteger(item.qty) || item.qty < 1);
  if (badItem) {
    return fail(`${raw.id}: পণ্যের তথ্য ভুল`);
  }
  const items = raw.items.map(({ sku, qty }) =>
    ({ sku: sku.trim(), qty, refunded: 0 }));
  const id = raw.id.trim();
  return ok({ id, customer: raw.customer.trim(), date: raw.date, items });
}
```

src/transform.mjs

দুই অংশ মিলে সম্পূর্ণ File।

transform.mjs · JavaScript — অংশ ১ / ২ (Part 1 of 2) — চলবে: Browser Console ও Node.js

```
import { updateIn } from './immutable.mjs';

const banglaCollator = new Intl.Collator('bn');

export const indexBy = (rows, key) => new Map(rows.map((row) => [row[key], row]));

// Order থেকে পণ্য-সারি; ফেরত বাদে বিক্রি পরিমাণ।
export const flattenOrders = (orders) =>
```

```
  orders.flatMap((order) => order.items.map((item) => ({
    order: order.id,
    customer: order.customer,
    month: order.date.slice(0, 7),
    sku: item.sku,
    qty: item.qty - item.refunded,
  })));

// পণ্য ও গ্রাহকের সঙ্গে Join; না মেলা সারি আলাদা।
export function joinLines(lines, products, customers) {
  const productIndex = indexBy(products, 'sku');
  const customerIndex = indexBy(customers, 'id');
  const joined = [];
  const unmatched = [];

  for (const line of lines) {
    const product = productIndex.get(line.sku);
    const customer = customerIndex.get(line.customer);
    if (!product || !customer) {
      unmatched.push(`${line.order}:${product ? line.customer : line.sku}`);
      continue;
    }
    joined.push({
      ...line,
      region: customer.region,
      customerName: customer.name,
      amount: product.price * line.qty,
    });
  }
  return { joined, unmatched };
}

// ...
```

transform.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser Console ও Node.js

```
// ...

const sum = (rows) => rows.reduce((total, row) => total + row.amount, 0);

// অঞ্চল × মাস; ঘরগুলো মাসের ক্রমে।
export function pivotRegionMonth(rows) {
  const pivot = {};
  for (const [region, regionRows] of Map.groupBy(rows, (row) => row.region)) {
    const months = Map.groupBy(regionRows, (row) => row.month);
    pivot[region] = Object.fromEntries(
      [...months].toSorted(([a], [b]) => a.localeCompare(b))
        .map(([month, monthRows]) => [month, sum(monthRows)]),
    );
```

```
  }
  return pivot;
}

export function topCustomers(rows, count = 3) {
  return [...Map.groupBy(rows, (row) => row.customerName)]
    .map(([name, customerRows]) => ({ name, total: sum(customerRows) }))
    .toSorted((a, b) =>
      b.total - a.total || banglaCollator.compare(a.name, b.name))
    .slice(0, count);
}

// একটি Order-এর একটি পণ্যের ফেরত পরিমাণ যোগ; বাকি সব অক্ষত।
export function applyRefund(orders, { orderId, sku, qty }) {
  const orderIndex = orders.findIndex((order) => order.id === orderId);
  const items = orders[orderIndex]?.items ?? [];
  const itemIndex = items.findIndex((item) => item.sku === sku);
  if (orderIndex === -1 || itemIndex === -1) {
    throw new RangeError(`${orderId}/${sku} পাওয়া যায়নি`);
  }
  const { qty: sold, refunded } = orders[orderIndex].items[itemIndex];
  if (!Number.isInteger(qty) || qty < 1 || refunded + qty > sold) {
    throw new RangeError(
      `ফেরত ${qty}টি সম্ভব নয়; বিক্রি ${sold}, আগে ফেরত ${refunded}`);
  }
  const path = [orderIndex, 'items', itemIndex, 'refunded'];
  return updateIn(orders, path, (n) => n + qty);
}

export const totalOf = sum;
```

- **`flattenOrders`** Nested Order থেকে সমতল পণ্য-সারি; একই সঙ্গে প্রতিটি সারিতে মাস (`date`-এর প্রথম সাত অক্ষর) এবং ফেরত বাদে নিট পরিমাণ।
- **`joinLines`** দুটি Index একবার তৈরি করে, তারপর প্রতিটি সারিতে দুটি `get`। অচেনা পণ্য বা গ্রাহক `unmatched`-এ, কারণসহ।
- **`pivotRegionMonth`** `Map.groupBy` দিয়ে দুই স্তরে ভাগ; মাসগুলো ক্রমে সাজানো, যাতে Report-এর কলাম নিশ্চিত ক্রমে আসে।
- **`topCustomers`**: মোট বেশি আগে; সমান হলে নাম বাংলা নিয়মে—নিশ্চিত ক্রম।
- **`applyRefund`** প্রথমে যাচাই (অচেনা Order বা পণ্য, অতিরিক্ত ফেরত), তারপর `updateIn`-এর একটি Path। Structural Sharing নিজে থেকে: শুধু সেই Order, তার `items` এবং সেই পণ্যটি নতুন।

src/load.mjs এবং নমুনা তথ্য

`load.mjs` · JavaScript — চলবে: Browser Console ও Node.js

```
// নাম ধরে File আনে, সর্বোচ্চ batchSize একসঙ্গে; ব্যর্থ File বাকিদের থামায় না।
```

```
export async function loadInBatches(names, loader, batchSize = 2) {
  const records = [];
  const failures = [];

  for (let i = 0; i < names.length; i += batchSize) {
    const batch = names.slice(i, i + batchSize);
    const results = await Promise.allSettled(batch.map((name) => loader(name)));
    results.forEach((result, j) => {
      if (result.status === 'fulfilled') {
        records.push(...result.value);
      } else {
        failures.push(`${batch[j]}: ${result.reason.message}`);
      }
    });
  }
  return { records, failures };
}
```

নমুনা তথ্যের File দুই অংশে; দুটি মিলে সম্পূর্ণ File।

sample.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Browser Console ও Node.js

```
export const PRODUCTS = [
  { sku: 'B1', name: 'Notebook', price: 120 },
  { sku: 'B2', name: 'Pen Set', price: 250 },
  { sku: 'B3', name: 'Atlas', price: 900 },
];

export const CUSTOMERS = [
  { id: 'C1', name: 'সারা', region: 'Dhaka' },
  { id: 'C2', name: 'রফিক', region: 'Bogura' },
  { id: 'C3', name: 'মিনা', region: 'Dhaka' },
];

// ...
```

sample.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Browser Console ও Node.js

```
// ...

const FILES = {
  '2026-01': [
    {
      id: 'O1', customer: 'C1', date: '2026-01-05',
      items: [{ sku: 'B1', qty: 2 }, { sku: 'B3', qty: 1 }],
    },
    {
      id: 'O2', customer: 'C2', date: '2026-01-09',
      items: [{ sku: 'B2', qty: 3 }],
```

```
    },
  ],
  '2026-02': [
    {
      id: 'O3', customer: 'C3', date: '2026-02-11',
      items: [{ sku: 'B3', qty: 1 }, { sku: 'B9', qty: 1 }],
    },
    {
      id: 'O4', customer: 'C1', date: '2026-02-30x',
      items: [{ sku: 'B1', qty: 1 }],
    },
  ],
  '2026-03': [
    {
      id: 'O5', customer: 'C2', date: '2026-03-02',
      items: [{ sku: 'B1', qty: 5 }],
    },
  ],
};

// কৃত্রিম Asynchronous Loader: অজানা মাসে ব্যর্থ।
export async function sampleLoader(name) {
  await new Promise((resolve) => setTimeout(resolve, 10));
  if (!Object.hasOwn(FILES, name)) {
    throw new Error('File নেই');
  }
  return structuredClone(FILES[name]);
}
```

report.mjs

report.mjs · JavaScript চলবে: Node.js

```
import { CUSTOMERS, PRODUCTS, sampleLoader } from './data/sample.mjs';
import { loadInBatches } from './src/load.mjs';
import { validateOrder } from './src/validate.mjs';
import {
  applyRefund, flattenOrders, joinLines, pivotRegionMonth, topCustomers, totalOf,
} from './src/transform.mjs';

const months = ['2026-01', '2026-02', '2026-03', '2026-04'];
const { records, failures } = await loadInBatches(months, sampleLoader);

const results = records.map(validateOrder);
const orders = results.filter((r) => r.ok).map((r) => r.value);
const invalid = results.filter((r) => !r.ok).map((r) => r.error);

const corrected = applyRefund(orders, { orderId: 'O5', sku: 'B1', qty: 2 });
const lines = flattenOrders(corrected);
```

```
const { joined, unmatched } = joinLines(lines, PRODUCTS, CUSTOMERS);
const pivot = pivotRegionMonth(joined);

console.log(`Order: ${records.length}, বৈধ ${orders.length};`,
  `বাদ: ${invalid.join('; ')}`);
console.log(`File ব্যর্থ: ${failures.join('; ')}; অমিল: ${unmatched.join(', ')}`);
console.log(pivot);
console.log(topCustomers(joined).map((c) => `${c.name} ${c.total}`).join(', '));

const pivotTotal = Object.values(pivot)
  .flatMap((months) => Object.values(months))
  .reduce((a, b) => a + b, 0);
console.log(`Invariant: ${pivotTotal === totalOf(joined)} (${pivotTotal})`);
console.log(corrected[0] === orders[0], corrected.at(-1) === orders.at(-1));
```

Output

```
Order: 5, বৈধ 4; বাদ: O4: তারিখ YYYY-MM-DD নয়
File ব্যর্থ: 2026-04: File নেই; অমিল: O3:B9
{
  Dhaka: { '2026-01': 1140, '2026-02': 900 },
  Bogura: { '2026-01': 750, '2026-03': 360 }
}
সারা 1140, রফিক 1110, মিনা 900
Invariant: true (3150)
true false
```

Code-এর ধাপে ধাপে ব্যাখ্যা

- **Load:** চারটি মাস, দুটি করে একসঙ্গে। এপ্রিলের File নেই; `allSettled` থাকায় সেটি ব্যর্থতার তালিকায়, বাকি তিনটির পাঁচটি Order হাতে।
- **Validate:** `O4`-এর তারিখ ভুল (`2026-02-30x`); Result Type তাকে কারণসহ আলাদা করে, Pipeline থামে না।
- **Refund:** `O5`-এর ৫টি Notebook থেকে ২টি ফেরত। শেষ লাইন দেখায়: প্রথম Order (`O1`) পুরোনো ও নতুন তালিকায় **একই** Object (Structural Sharing), শুধু `O5` নতুন।
- **Flatten ও Join:** `O3`-এর `B9` পণ্যটি তালিকায় নেই; বাকি সারি থাকে, অমিলটি জানানো হয়।
- **Pivot:** ঢাকার জানুয়ারি = সারার Order: ২×১২০ + ৯০০ = ১১৪০; বগুড়ার মার্চ = ফেরতের পর ৩টি Notebook = ৩৬০।
- **শীর্ষ গ্রাহক** ও **Invariant:** Pivot-এর সব ঘরের যোগফল সব সারির মোটের সমান (৩১৫০)—কোনো সারি হারায়নি বা দুবার গোনা হয়নি।

চালানো এবং প্রত্যাশিত আচরণ

```
Terminal
node report.mjs
node --test
```

Companion Source Code-এ ২২টি Test, প্রতিটি ইনপুট Deep Freeze করে চালানো: Structural Sharing (`===` যাচাই), `updateIn`-এর Array ও Object পথ, যাচাইয়ের প্রতিটি ব্যর্থতা, Join-এর অমিল, Pivot-এর Invariant, শীর্ষ গ্রাহকের নিশ্চিত ক্রম, অতিরিক্ত ফেরতের প্রত্যাখ্যান, এবং Batch Loader-এর সর্বোচ্চ Concurrency ও আংশিক ব্যর্থতা (Fake Timer ছাড়াই, নিয়ন্ত্রিত Loader দিয়ে)।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Refund-এর পর মূল তালিকাও বদলেছে	Nested Spread-এ একটি স্তর বাদ	`updateIn`
Pivot-এর যোগফল মোটের চেয়ে কম	Join-এ সারি নিঃশব্দে বাদ	`unmatched` দেখুন
শীর্ষ গ্রাহকের ক্রম মাঝে মাঝে বদলায়	সমান মোটে দ্বিতীয় Key নেই	নাম ধরে Collator
একটি File ব্যর্থ হলে সব থামে	`Promise.all`	`Promise.allSettled`
Test-এ `TypeError: Cannot assign`	কোনো Function ইনপুট বদলাচ্ছে	সেই Function Immutable করুন

এই Project-এর সীমাবদ্ধতা

এটি একটি শিক্ষামূলক Project। তথ্য Memory-তে; লক্ষাধিক সারির জন্য Stream বা Database লাগবে। মাস ধরা হয়েছে তারিখের লেখা থেকে, সময়-অঞ্চল বিবেচনা ছাড়া। টাকা পূর্ণসংখ্যায়।

১৮.১৪ অনুশীলনী

প্রাথমিক স্তর

১. নিচের কোনটি Referentially Transparent? `(x) => x * 2`, `() => Date.now()`, `(list) => list.length`। কারণ বলুন।

২. `{ user: { profile: { city: 'Bogura' } } }`-এ শহর `Dhaka` করুন Spread দিয়ে, মূলটি অক্ষত রেখে।

৩. `flatMap` দিয়ে `['a,b', 'c']` থেকে `['a', 'b', 'c']` তৈরি করুন।

৪. `Map.groupBy` দিয়ে সংখ্যাগুলোকে জোড় ও বিজোড়ে ভাগ করুন, Key `true` / `false` (Boolean)।

৫. বাংলা নামের একটি তালিকা `Intl.Collator('bn')` দিয়ে সাজান।

মধ্যম স্তর

৬. `updateIn` দিয়ে একটি Nested Array-র (`matrix[1][2]`) একটি মান দ্বিগুণ করুন, এবং দেখান অন্য সারিগুলো একই Object।

৭. দুটি তালিকা (শিক্ষার্থী ও ফলাফল) ID ধরে Left Join করুন: ফল না থাকলে `marks: null`।

৮. একটি `reduce`-এ বিক্রির তালিকা থেকে সংখ্যা, মোট, গড় ও সর্বনিম্ন বের করুন।

৯. Result Type-এ একটি `map` সাহায্যকারী লিখুন, যা সফল Result-এর মান রূপান্তর করে, ব্যর্থতা অপরিবর্তিত রাখে।

চ্যালেঞ্জ স্তর

১০. `diffShallow(before, after)` লিখুন, যা দুটি Object-এর কোন Key-গুলোর মান `===` অনুযায়ী বদলেছে তা দেয়; Structural Sharing-এর সুবিধা দেখান।

১১. একটি `asyncFilter(items, predicate)` লিখুন, যা Asynchronous শর্তে সঠিকভাবে ছাঁকে।

১২. LedgerFlow-এ পণ্যভিত্তিক মোট বিক্রি (নাম ও পরিমাণ) এবং সবচেয়ে বেশি বিক্রি হওয়া পণ্য যোগ করুন।

১৮.১৫ অনুশীলনীর সমাধান

সমাধান ১ — `(x) => x * 2` হ্যাঁ: যেকোনো `x`-এ ফল দিয়ে প্রতিস্থাপন করা যায়। `() => Date.now()` না: প্রতি ডাকে আলাদা মান। `(list) => list.length` তালিকাটি বদলানো না হলে হ্যাঁ; কিন্তু একই তালিকা পরে বদলালে একই Argument-এ আলাদা ফল। এজন্যই Immutable তথ্যের সঙ্গে Referential Transparency স্বাভাবিকভাবে আসে।

সমাধান ২

nested-city.js · JavaScript — চলবে: Browser Console ও Node.js

```
const state = {
  user: { profile: { city: 'Bogura', zip: '5800' }, name: 'Sara' },
};

const next = {
  ...state,
  user: { ...state.user, profile: { ...state.user.profile, city: 'Dhaka' } },
};

const { city, zip } = next.user.profile;
console.log(state.user.profile.city, city, zip);
```

Output

```
Bogura Dhaka 5800
```

সমাধান ৩

flat-split.js · JavaScript চলবে: Node.js

```
console.log(['a,b', 'c'].flatMap((text) => text.split(',')));
```

Output

```
[ 'a', 'b', 'c' ]
```

সমাধান ৪

even-odd.js · JavaScript চলবে: Node.js

```
const groups = Map.groupBy([1, 2, 3, 4, 5], (n) => n % 2 === 0);

console.log(groups.get(true), groups.get(false), [...groups.keys()]);
```

Output

```
[ 2, 4 ] [ 1, 3, 5 ] [ false, true ]
```

Key-গুলো Boolean রয়ে গেছে; ক্রম প্রথম দেখা অনুযায়ী (`1` বিজোড়, তাই `false` আগে)।

সমাধান ৫

bangla-sort.js · JavaScript চলবে: Node.js

```
const names = ['রফিক', 'অনিক', 'তানিয়া', 'ইমরান', 'খাদিজা'];

console.log(names.toSorted(new Intl.Collator('bn').compare));
```

Output

```
[ 'অনিক', 'ইমরান', 'খাদিজা', 'তানিয়া', 'রফিক' ]
```

সমাধান ৬

matrix.mjs · JavaScript চলবে: Node.js

```
import { updateIn } from '../../project/ledger-flow/src/immutable.mjs';

const matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]];
const doubled = updateIn(matrix, [1, 2], (value) => value * 2);

console.log(doubled[1], matrix[1]);
const same = [0, 1, 2].map((row) => doubled[row] === matrix[row]);
console.log(same.join(' '));
```

Output

```
[ 4, 5, 12 ] [ 4, 5, 6 ]
true false true
```

সমাধান ৭

left-join.js · JavaScript চলবে: Node.js

```
const students = [
  { id: 1, name: 'Sara' }, { id: 2, name: 'Rafi' }, { id: 3, name: 'Mina' },
];
const results = [{ studentId: 1, marks: 88 }, { studentId: 3, marks: 72 }];

const marksById = new Map(results.map((r) => [r.studentId, r.marks]));
const report = students.map((s) =>
  ({ ...s, marks: marksById.get(s.id) ?? null }));

console.log(report.map((r) => `${r.name}:${r.marks}`).join(' '));
```

Output

```
Sara:88 Rafi:null Mina:72
```

Left Join বাম তালিকার সব সারি রাখে; ডান পাশে না মিললে খালি মান। Inner Join হলে Rafi বাদ পড়তেন।

সমাধান ৮

stats.js · JavaScript চলবে: Node.js

```
const amounts = [1200, 800, 500, 300];

const stats = amounts.reduce(
  (s, amount) => ({
    count: s.count + 1,
    total: s.total + amount,
    min: Math.min(s.min, amount),
  }),
  { count: 0, total: 0, min: Infinity },
);

console.log({ ...stats, average: stats.count ? stats.total / stats.count : 0 });
```

Output

```
{ count: 4, total: 2800, min: 300, average: 700 }
```

গড় `reduce`-এর পরে, মোট ও সংখ্যা থেকে; Reducer-এর ভেতরে প্রতিবার গড় হিসাব অপ্রয়োজনীয় এবং দশমিকের ভুল জমায়।

সমাধান ৯

result-map.js · JavaScript চলবে: Browser Console ও Node.js

```
const ok = (value) => ({ ok: true, value });
const fail = (error) => ({ ok: false, error });
const mapResult = (fn) => (result) => (result.ok ? ok(fn(result.value)) : result);

const toTaka = mapResult((n) => `${n} টাকা`);

console.log(JSON.stringify(toTaka(ok(250))));
console.log(JSON.stringify(toTaka(fail('দাম নেই'))));
```

Output

```
{"ok":true,"value":"250 টাকা"}
{"ok":false,"error":"দাম নেই"}
```

`map` মানের রূপান্তর (যা ব্যর্থ হতে পারে না), `andThen` এমন ধাপ যা নিজেই ব্যর্থ হতে পারে—দুটি আলাদা সাহায্যকারী।

সমাধান ১০

diff.mjs · JavaScript চলবে: Node.js

```
import { setIn } from '../../project/ledger-flow/src/immutable.mjs';

const diffShallow = (before, after) =>
  [...new Set([...Object.keys(before), ...Object.keys(after)])]
    .filter((key) => before[key] !== after[key]);

const state = { cart: { items: [1, 2] }, user: { name: 'Sara' }, theme: 'light' };
const next = setIn(state, ['cart', 'items'], [1, 2, 3]);

console.log(diffShallow(state, next));
console.log(diffShallow(state, setIn(state, ['theme'], 'light')));
```

Output

```
[ 'cart' ]
[]
```

গভীর তুলনা ছাড়াই, শুধু `===`-এ, বোঝা গেল শুধু `cart` বদলেছে; একই মান বসালে `setIn` পুরোনো Object-ই ফেরত দেয়, তাই কিছুই বদলায়নি। UI Library-গুলো এভাবেই ঠিক করে কোন অংশ নতুন করে আঁকবে।

সমাধান ১১

async-filter.mjs · JavaScript চলবে: Browser Console ও Node.js

```
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));
const isAvailable = async (sku) => {
  await wait(10);
  return sku !== 'B2';
};

async function asyncFilter(items, predicate) {
  const keep = await Promise.all(items.map(predicate));
  return items.filter((_, index) => keep[index]);
}

const wrong = ['B1', 'B2', 'B3'].filter(isAvailable);
console.log(wrong.length);
console.log((await asyncFilter(['B1', 'B2', 'B3'], isAvailable)).join(','));
```

Output

```
3
B1,B3
```

`filter(isAvailable)` প্রতিটি উপাদানের জন্য একটি Promise পায়, যা সব সময় Truthy, তাই কিছুই বাদ পড়েনি। আগে সব শর্তের ফল (`Promise.all`), তারপর সাধারণ `filter`।

সমাধান ১২

by-product.mjs · JavaScript চলবে: Node.js

```
const P = '../../project/ledger-flow';
const sample = await import(`${P}/data/sample.mjs`);
const { PRODUCTS, CUSTOMERS, sampleLoader } = sample;
const { loadInBatches } = await import(`${P}/src/load.mjs`);
const { validateOrder } = await import(`${P}/src/validate.mjs`);
const { flattenOrders, joinLines } = await import(`${P}/src/transform.mjs`);

const months = ['2026-01', '2026-02', '2026-03'];
const { records } = await loadInBatches(months, sampleLoader);
const orders = records.map(validateOrder).filter((r) => r.ok).map((r) => r.value);
const { joined } = joinLines(flattenOrders(orders), PRODUCTS, CUSTOMERS);
const names = new Map(PRODUCTS.map((p) => [p.sku, p.name]));

const bySku = [...Map.groupBy(joined, (row) => row.sku)]
  .map(([sku, rows]) => ({
    name: names.get(sku),
    qty: rows.reduce((a, r) => a + r.qty, 0),
  }))
  .toSorted((a, b) => b.qty - a.qty);

console.log(bySku.map((p) => `${p.name} ${p.qty}`).join(', '));
console.log(`সবচেয়ে বেশি: ${bySku[0].name}`);
```

Output

```
Notebook 7, Pen Set 3, Atlas 2
সবচেয়ে বেশি: Notebook
```

বিদ্যমান ধাপগুলো পুনর্ব্যবহার হয়েছে; শুধু শেষে একটি নতুন Grouping। এখানে Refund প্রয়োগ হয়নি, তাই `05`-এর ৫টি Notebook পুরোটাই গোনা।

১৮.১৬ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. Referential Transparency মানে কী?

ক) Function দ্রুত চলে

খ) Expression-কে তার ফল দিয়ে প্রতিস্থাপন করলে আচরণ বদলায় না

গ) Function-এর নাম নেই

ঘ) Code সবার জন্য দৃশ্যমান

প্রশ্ন ২. Structural Sharing-এ Nested তথ্যের একটি মান বদলালে কী নতুন তৈরি হয়?

ক) পুরো গাছ

খ) কিছুই না, মূলটি বদলায়

গ) শুধু বদলের পথের Object-গুলো

ঘ) শুধু বদলানো মানটি

প্রশ্ন ৩. `[[1], [2, 3]].flatMap((a) => a)`-এর ফল?

ক) `[1, 2, 3]`

খ) `[[1], [2, 3]]`

গ) `[1, [2, 3]]`

ঘ) `6`

প্রশ্ন ৪. Grouping-এর Key সংখ্যা হিসেবে রাখতে কোনটি?

ক) `Object.groupBy`

খ) `JSON.stringify`

গ) `Array.from`

ঘ) `Map.groupBy`

প্রশ্ন ৫. বড় তালিকায় Join দ্রুত করতে কী করবেন?

ক) প্রতিটি সারিতে `find`

খ) আগে একটি `Map` Index

গ) দুটি তালিকা সাজানো

ঘ) `structuredClone`

প্রশ্ন ৬. `toSorted` Stable—এর অর্থ?

ক) সমান উপাদানের মূল ক্রম অক্ষত থাকে

খ) মূল Array বদলায় না

গ) সব সময় একই সময় লাগে

ঘ) শুধু সংখ্যা সাজায়

প্রশ্ন ৭. বাংলা নাম অভিধানের ক্রমে সাজাতে কোনটি উপযুক্ত?

ক) `sort()` Argument ছাড়া

খ) `<` তুলনা

গ) `Intl.Collator('bn').compare`

ঘ) `reverse()`

প্রশ্ন ৮. `andThen` একটি ব্যর্থ Result পেলে কী করে?

ক) পরের Function চালায়

খ) ব্যর্থ Result অপরিবর্তিত ফেরত দেয়

গ) Exception ছোড়ে

ঘ) `null` দেয়

প্রশ্ন ৯. `items.filter(async (x) => ...)` কেন কাজ করে না?

ক) `filter`-এ Function দেওয়া যায় না

খ) `async` নিষিদ্ধ

গ) প্রতিটি Promise Truthy, তাই কিছুই বাদ পড়ে না

ঘ) সব বাদ পড়ে

প্রশ্ন ১০. অনেক File একসঙ্গে কিন্তু সীমিত সংখ্যায় আনতে, এবং একটির ব্যর্থতায় বাকিগুলো না থামাতে কোনটি?

ক) Batch এবং `Promise.allSettled`

খ) সব একসঙ্গে `Promise.all`

গ) `Promise.race`

ঘ) `forEach` ও `await`

১৮.১৭ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–গ · ৩–ক · ৪–ঘ · ৫–খ · ৬–ক · ৭–গ · ৮–খ · ৯–গ · ১০–ক

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	Pure Function-এর ডাক তার ফলের সমতুল্য।
২	গ	বাকি শাখা পুরোনো ও নতুন অবস্থা ভাগ করে নেয়।
৩	ক	`flatMap` এক স্তর সমতল করে।
৪	ঘ	`Map` -এর Key যেকোনো ধরনের; `Object.groupBy` লেখায় রূপান্তর করে।
৫	খ	Index একবার তৈরি, প্রতিটি খোঁজা তাৎক্ষণিক।
৬	ক	তাই বহু-Key সাজানো নির্ভরযোগ্য।
৭	গ	ভাষার নিয়মে তুলনা।
৮	খ	প্রথম ব্যর্থতার পরের ধাপগুলো বাদ পড়ে।
৯	গ	`filter` Promise-এর জন্য অপেক্ষা করে না।
১০	ক	দল চাপ সীমিত রাখে; `allSettled` প্রতিটির ফল দেয়।

১৮.১৮ অধ্যায়ের সারসংক্ষেপ

- Referentially Transparent Code-এ Refactor ও Memoization নিরাপদ।
- Nested তথ্য হালনাগাদ শুধু বদলের পথ ধরে; বাকি শাখা ভাগ করা (Structural Sharing), পরিবর্তন `===` -এ ধরা যায়।
- `updateIn` ও `setIn` Path ধরে হালনাগাদ সহজ করে; Deep Freeze Test-এ Mutation ধরে।
- `flatMap` Nested তথ্য সমতল করে; `reduce` দিয়ে `Map` -এ জমানো।
- `Map.groupBy` Key-র ধরন রাখে; বহুস্তর Grouping-এ Pivot; একটি `reduce` -এ একাধিক Aggregate।
- Join-এর আগে `Map` Index; না মেলা সারি জানান।
- সাজানো Stable; বহু-Key Comparator; বাংলায় `Intl.Collator`।
- Result Type ব্যর্থতাকে মান হিসেবে প্রবাহিত করে।
- Asynchronous Pipeline: ক্রমে, একসঙ্গে, বা দলে দলে; `map` / `filter` Promise-এর জন্য অপেক্ষা করে না।

পরের অধ্যায়ে Performance Optimization ও Memory Management: এই অধ্যায়ের Index, Cache ও Batch-এর সিদ্ধান্তগুলো কীভাবে মেপে নিতে হয়।

১৮.১৯ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Referential Transparency	Expression-কে ফল দিয়ে প্রতিস্থাপনযোগ্যতা।

পরিভাষা	অর্থ
Structural Sharing	নতুন ও পুরোনো অবস্থার অপরিবর্তিত অংশ ভাগাভাগি।
Deep Freeze	সব স্তরে অপরিবর্তনীয় করা।
`flatMap`	Map করে এক স্তর সমতল করা।
Aggregation	একটি দলের সংখ্যা, মোট, সর্বোচ্চ ইত্যাদি।
Pivot	দুই মাত্রার Grouping-এর সারণি।
Index	Key থেকে সারিতে দ্রুত পৌঁছানোর `Map`।
Inner / Left Join	উভয় পাশে মেলা সারি / বাম পাশের সব সারি।
Stable Sort	সমান উপাদানের মূল ক্রম রক্ষাকারী সাজানো।
Collator	ভাষার নিয়মে লেখা তুলনাকারী।
Result Type	সফল মান বা ব্যর্থতা বহনকারী মান।
Batch	একসঙ্গে প্রক্রিয়া করা কাজের দল।

১৮.২০ আরও পড়ুন

- **MDN — Array.prototype.flatMap():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/flatMap` — `flatMap`।
- **MDN — Map.groupBy():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Map/groupBy` — Key-র ধরন রেখে Grouping।
- **MDN — Intl.Collator:** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Collator` — ভাষাভিত্তিক তুলনা।
- **MDN — Array.prototype.sort():** `https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/sort` — Stability ও Comparator।
- **Immer — Introduction:** `https://immerjs.github.io/immer/` — Immutable হালনাগাদের একটি জনপ্রিয় Library।